Kayetan Jarzabek
CS498
HW3

Part 1: Wide-Column Schema Design & Sharding

1) **Row Key & Sharding:** For the EV dataset, I would use a row key which would hold a hashed DOL vehicle ID followed by the actual DOL vehicle ID sort of like: hash(DOL Vehicle ID)%16 # DOL Vehicle ID. This would help prevent any bottlenecks in too many writes or reads by helping distribute the vehicle records across the different key ranges rather than concentrating all the writes in one location. However, querying by city or make would be less efficient since neither is really included in the row key and, as such, separate lookup tables could be used to improve these queries.

2) **Column Families:** I would organize the attributes into three column families and they would like this:

Vehicle: VIN, make, model, model year, and EV type
Location: County, city, state, postal code, legislative district, vehicle location, GEOID
Energy: Electric range, electric utility, CAFV eligibility

By grouping the related attributes together allows Bigtable to retrieve them by only the column families needed for a query which helps reduce any unnecessary data reads and therefore improving overall performance.

3) **Horizontal Scaling:** A hashed shard key for the VIN would help distribute records more evenly across the nodes and thus reduce the bottlenecks in either too many writes or reads and improve overall performance. However, range queries become less efficient because the records are found across multiple different partitions. A range-based shard key for the model year makes the year-based queries more efficient because the vehicles from similar years are stored together. However, it can cause a bottleneck in either too many writes or reads when many of the records have the same recent model year and thus creating an uneven workload across the nodes. Overall, hashed sharding is better for distributing the workloads evenly while range-based sharding is better for queries that have specific ranges of model years.

Part 4: Performance Analysis

1) I had a small Python benchmark script that sent 500 POST requests to /insert-fast and 500 POST requests to /insert-safe in order to compare the two write concerns. From the script, I was able to get the response time of each request and then calculate the average latency for both.

2) **For fast write (w=1):** There was an average latency of 18.56ms and a median latency of 6.35ms.
For safe write (w="majority"): There was an average latency of 20.13ms and a median latency of 8.82ms

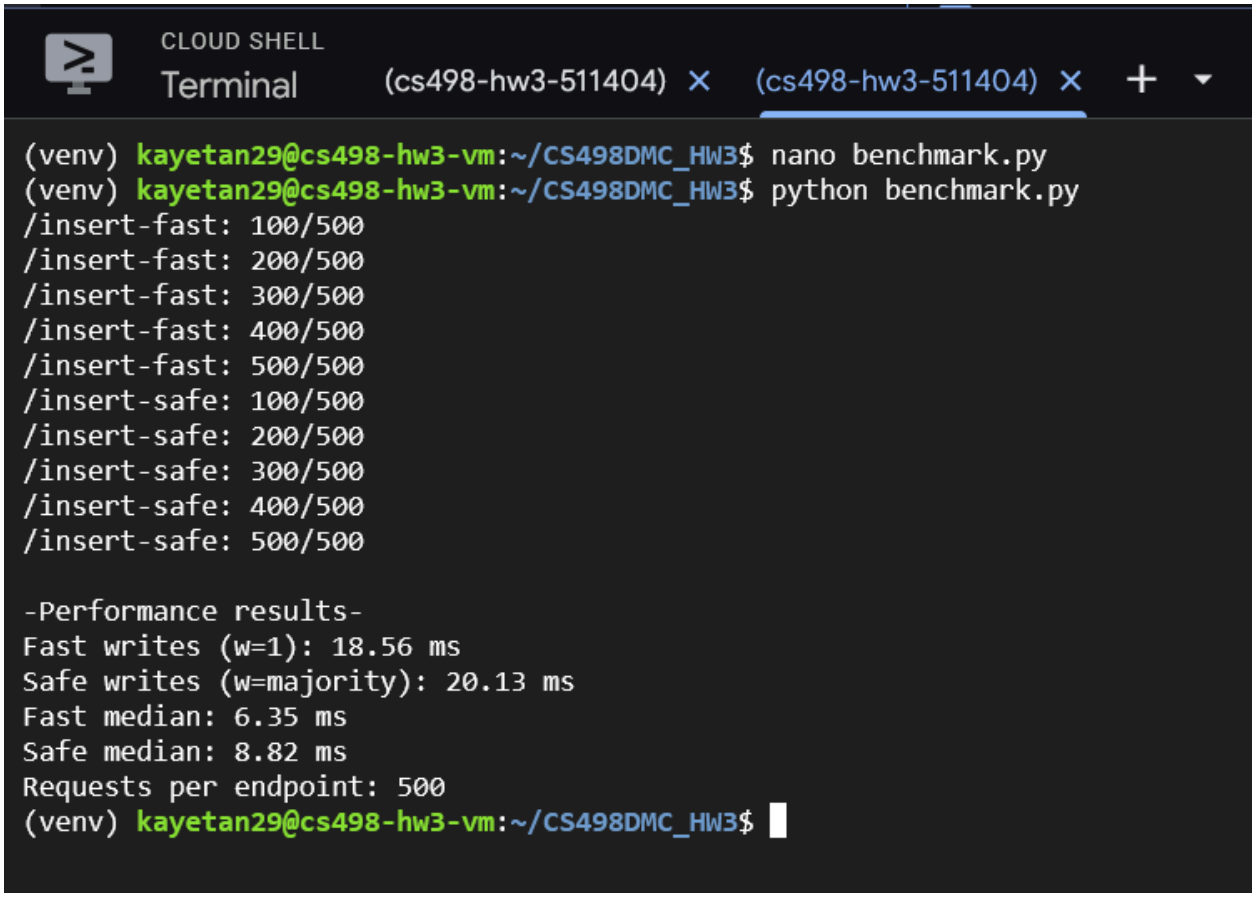


Figure 1: My Python's benchmark of 500 (w=1) and 500 (w="majority") writes

The fast writes had an average latency of 18.56ms compared to the safe write's 20.13ms revealing that they are slightly faster. This makes sense to me because the fast writes only require an acknowledgement from the primary node while the safe writes need an acknowledgement from a majority.

3) **Analysis Question:** A real-world scenario where I would choose the faster primary only write concern would have to be an application where we have to collect any noncritical data like logs of user activity or analytics for a website. In these cases, having a faster response time might be more important than making sure every write is saved. If the primary node crashes then we would lose some data/records but this wouldn't affect the app's overall function. In relation to the CAP theorem, this shows a tradeoff between the performance and stronger consistency

guarantees in a distributed database. When relating it to the CAP theorem this shows how distributed database systems have to consider tradeoffs between availability and performance. With w=1, the database can acknowledge writes faster with weaker durability guarantees while w="majority" has stronger durability by requiring acknowledgment from multiple members. However, write concerns primarily control acknowledgment and durability rather than directly determining a system's CAP classification.